

Chapter 0

과정 소개와 실습 환경 준비

Observe → Understand → Implement → Analyze
2 Unit · 영상 16 분 · VirtualBox + Ubuntu Server 26.04

```
$ sudo ./env_check.sh
[ OK ] Ubuntu 26.04 LTS
[ OK ] Kernel 7.0 이상
[ OK ] CPU 2 개
[WARN] Hardware Counter 사용 불가
...
== 결과 : OK 29 / WARN 1 / FAIL 0
실습 준비 완료 .
```

man page 만으로는 답하기 어려운 질문

- 1 `write()` 가 `len` 을 `return` 한 직후 전원이 꺼지면 `Data` 는 남을까 ? Ch3 L15
- 2 `read()` 는 항상 `Disk` 를 읽을까 ? Ch3 L13
- 3 `pthread_mutex_lock()` 은 호출할 때마다 `Kernel` 에 들어갈까 ? Ch4 L19
- 4 `malloc(1GB)` 직후 `Memory` 사용량이 거의 늘지 않는 이유는 ? Ch2 L09
- 5 프로그램이 느리다 . 무엇부터 조사할까 ? Ch7

넘어가기 전에 각 질문의 답을 한 줄씩 적어 두세요 . 과정이 끝난 뒤 다시 비교합니다 . 답은 `System Call` 경계 아래 , `Kernel` 안에 있습니다 .

" 어떻게 쓰는가 " 에서 " 왜 이렇게 동작하는가 " 로

	기존 과정	이번 과정
질문	어떻게 사용하는가	왜 이렇게 동작하는가
범위	API 와 사용 Pattern	System Call 아래 Kernel 동작 , CPU 와 Memory
확인 방법	실행 결과	strace · /proc · perf · ftrace · bpftrace 로 직접 관찰
execve	호출하면 프로그램이 교체된다	ELF 가 어떻게 Mapping 되고 main() 까지 가는가
mutex	lock / unlock 사용법	언제 futex 로 Kernel 에 진입하는가
read / write	fd 와 buffer	VFS · Page Cache · Writeback · fsync

가장 큰 차이는 확인 방법 : 설명을 듣고 끝내지 않고 , 설명한 내용이 사실인지 도구로 직접 확인한다 .

Process 단면도로 보는 8 Chapter

Application · glibc

Ch1 실행 · System Call · Ch4 Atomic · Ch5 Event Loop

Shared Ring

Ch6 io_uring (SQ / CQ)

System Call 경계

Ch1 syscall · ELF · Loader

Kernel

Ch2 Memory · Ch3 VFS · Page Cache · Ch4 futex · Ch5
eventpoll

Hardware

Ch2 TLB · Ch3 Storage · Ch4 Cache Line

▶ 애니메이션 [C0-1 Process 단면도로 보는 8 Chapter](#)

Ch7 · 도구

모든 층을 옆에서 들여다본다
/proc · strace · perf
ftrace · bpftrace

Ch8 · 울타리

Process 바깥의 제한
Namespace · Cgroup
Capability · seccomp

Network Programming 은 별도 과정에서

다루지 않는다

socket · bind · listen · accept

TCP / UDP · HTTP

Network Client / Server

→ Linux Network System Programming 과정

다룬다 (범용 FD 메커니즘으로)

select · poll · epoll

여러 File Descriptor 를 한꺼번에 기다리는 방법

Network API 가 아니다

```
# epoll 실습에 socket 대신 사용하는 FD
pipe    FIFO    eventfd    timerfd    signalfd    inotify
```

Lecture 한 편의 흐름과 자료 구성



권장 순서 : 영상 → lecture.html 따라 하기 (출력 비교 , 다르면 오류 해결 표) → animation.html 직접 조작 → 생각해 볼 질문 → worksheet.html

Ubuntu Server 26.04 LTS · Linux 7.0

구성	Version	확인 명령
Host	Windows 11	
VirtualBox	7.2.8 이상	도움말 > VirtualBox 정보
Guest OS	Ubuntu Server 26.04 LTS	lsb_release -ds
Kernel	Linux 7.0 (7.0.0-xx-generic)	uname -r
Compiler	GCC 15	gcc --version
접속	PuTTY (SSH) · FileZilla (SFTP)	127.0.0.1:2222

Server ISO 를 받는다

Desktop 이 아니라
ubuntu-26.04.x-live-server-
amd64.iso

모든 실습은 SSH Terminal 에서 하고,
Server 가 Memory 를 훨씬 적게 쓴다.

Ubuntu 26.04.1 처럼 Point Release 가 붙어도 같은 26.04 LTS. Kernel 의 -xx 는 Update 에 따라 달라지고 앞의 7.0 만 같으면 된다.

Processor 는 반드시 2 개 이상

항목	값
이름	lsp2-ubuntu2604
무인 설치	사용하지 않음
Memory	4096 MB 이상
Processor	2 개 이상
Hard Disk	40 GB (동적 할당)
Network	NAT

2+

Processor

1 개면 Ch4 Race 재현 불가

Port Forwarding

ssh · TCP
127.0.0.1 : 2222
→ Guest : 22

설치 중 반드시 : Install OpenSSH server 에 Check (Space 로 선택). 빠뜨리면 PuTTY 로 접속할 수 없다 .

```
$ lsb_release -ds; uname -r; nproc
Ubuntu 26.04.1 LTS
7.0.0-14-generic
2
```

관찰이 핵심인 과정은 도구가 먼저

```
$ git clone https://github.com/imguru-mooc/adv-sysprog.git
~/lsp2
$ cd ~/lsp2/ch00/src
$ sudo ./install_tools.sh
[ OK ] build-essential
[ OK ] strace ...
[ OK ] linux-tools-7.0.0-14-generic
```

도구	Chapter
gcc · make · strace	전체
ltrace · gdb	1 · 7
perf (linux-tools)	2 · 4 · 7
trace-cmd · bpftrace	7
liburing	6
libseccomp · libcap	8

실습은 항상 **Home Directory** 아래에서 .

Ubuntu 26.04 의 /tmp 는 Memory 위의 tmpfs. /tmp 에서 File I/O 실습을 하면 Ch3 Page Cache · Writeback · fsync 결과가 전혀 다르게 나온다 .

env_check.sh: FAIL 0 이면 준비 완료

[OK]

준비 완료
→ 할 일 없음

[WARN]

실습 가능, 일부 결과가 강의와 다를 수 있음
→ 설명을 읽고 넘어감

[FAIL]

해결해야 함
→ 아래 줄 -> 뒤의 명령 실행

Event 종류	누가 세는가	예	VirtualBox
Software Event	Kernel	page-faults, context-switches	사용 가능
Hardware Event	CPU 안의 PMU	cycles, cache-misses	대부분 not supported

Hardware Counter WARN 은 정상 .
기본 실습은 Software Event 로
구성했고 , cache-misses 가 필요한 장면
(L08 · L20 · L33) 은 Native Linux
결과를 함께 실행한다 .

▶ 애니메이션 [C0-2 환경 점검 시뮬레이터](#)

같은 프로그램, 도구마다 보이는 층이 다르다

strace

System Call

```
write(1, "hello, kernel\n",  
14)  
write(1, "pid=... touched=256  
...", 37)
```

ltrace

Library 함수

```
malloc(1048576)  
memset(..., 1, 1048576)  
free(...)
```

perf stat

Kernel Event

```
318 page-faults  
0 context-switches
```

bpftrace

Kernel 안에서

```
write fd=1 len=14  
write fd=1 len=37
```

hello_trace 가 하는 일 : **write** 한번 · **malloc** 한번 · **Memory 256 Page** 에 쓰기

page-faults 가 256 보다 큰 이유 : **Page** 를 처음 건드리는 순간마다 **Fault** + 프로그램을 올리는 과정의 **Fault** (Ch2)

이전 Ubuntu 와 달라진 것

변화	이 과정에 미치는 영향
Linux Kernel 7.0	io_uring · eBPF · cgroup v2 를 추가 설정 없이 사용
GCC 15 (기본 C23)	제공 Source 는 그대로 Build. 오래된 코드는 bool · 빈 괄호 선언에서 오류 가능
sudo-rs	Password 입력 시 * 표시 . 사용법 동일
/tmp 가 tmpfs	File I/O 실습은 반드시 Home Directory 아래에서 (Ch3)
cgroup v1 제거	cgroup v2 만 존재 . Ch8 은 v2 기준
Rust 기반 coreutils	ls · cat · stat 사용법 동일

마지막 단계 : VM 종료 후 Snapshot tools-installed. root 권한으로 System 설정을 바꾸는 실습 (drop_caches · ftrace · cgroup · namespace) 이 많으므로 언제든지 되돌릴 수 있게 한다 .

프로그램 실행과 System Call 내부 구조

- ☐ `lsb_release -ds` 가 Ubuntu 26.04, `uname -r` 이 7.0 으로 시작한다
- ☐ `nproc` 이 2 이상이다
- ☐ `sudo ./env_check.sh` 결과 FAIL 이 0 개다
- ☐ `strace` · `ltrace` · `perf stat` · `bpftool` 로 `hello_trace` 를 관찰했다
- ☐ `Snapshot tools-installed` 를 만들었다

다음 : Chapter 1 · L01

방금 `strace` 에서 본 한 줄
`write(1, "hello, kernel\n", 14) = 14`
이 출력되기까지 CPU 안에서는 무슨 일이 있었을까 ?

[worksheet.html](#)

20 문항

개념 워크시트

[animation.html](#)

2 종

애니메이션

[lecture.html](#)

18 칸

빈칸 · 괄호

저장소 github.com/imguru-mooc/adv-sysprog · 온라인 자료 imguru-mooc.github.io/adv-sysprog